Flow-Sensitive Static Analysis for Post-Quantum Cryptography Migration: A Risk-Based Approach

Aaryan Shelar
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
aaryan.shelar@vit.edu

Sarthak Bhat
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
sarthak.bhat@vit.edu

Sarvesh Alegaonkar
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
sarvesh.alegaonkar@vit.edu

Ramjan Shaikh
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
ramjan.shaikh@vit.edu

Jagruti Sarode
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
jagruti.sarode@vit.edu

Prof. Ashlesha Sawant
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
ashlesha.sawant@vit.edu

***Abstract*—The advent of large-scale quantum computers poses a critical and imminent threat to classical public-key cryptography (PKC), including widely utilized algorithms such as RSA and Elliptic Curve Cryptography (ECC). This vulnerability is primarily driven by Shor's algorithm, which can theoretically break these cryptographic primitives in polynomial time. Consequently, governments and organizations worldwide are mandated to migrate their digital infrastructure to Post-Quantum Cryptography (PQC). A paramount challenge in this transition is "cryptographic discovery"—the identification and prioritization of vulnerable cryptographic assets that are often deeply embedded, undocumented, and scattered across massive legacy codebases. Traditional baseline scanners rely heavily on regular expressions and lexical text matching. While fast, these tools suffer from unacceptably high false positive rates due to their inability to understand code semantics, context, or data flow. In this paper, we present the PQC Migration Analyzer Platform, an advanced, research-grade tool designed for automated, highly accurate cryptographic discovery in Java repositories. Our platform employs an Abstract Syntax Tree (AST)-based, interprocedural, and flow-sensitive static analysis engine. By implementing sophisticated taint tracking and structural exposure calculations, our tool not only detects primitive cryptographic API calls but also evaluates how cryptographic keys and data propagate through the application across method boundaries. Furthermore, we introduce a novel Quantum Risk Score (QRS) that mathematically quantifies the vulnerability level of identified assets, enabling prioritized remediation based on actual risk rather than mere presence. Extensive experimental evaluation against a standard regex-based baseline scanner demonstrates that our approach significantly improves precision (reducing false positives) and recall, while effectively ranking the most critical vulnerabilities at the top. Our platform provides a scalable, accurate, and actionable solution to accelerate PQC migration readiness in enterprise software environments.**

***Index Terms*—Post-Quantum Cryptography, Static Code Analysis, Taint Analysis, Shor's Algorithm, Software Security, Quantum Risk Score, Java Security, Cryptographic Discovery.**

I. Introduction

A. The Impending Quantum Threat

The rapid and continuous advancement of quantum computing technology threatens to undermine the foundational security of the modern digital infrastructure. Since the inception of the internet, secure communication, digital signatures, and data confidentiality have relied on classical public-key cryptography (PKC). Algorithms such as the Rivest-Shamir-Adleman (RSA) algorithm and Elliptic Curve Cryptography (ECC) base their security on the presumed intractability of specific mathematical problems: integer factorization and the discrete logarithm problem. However, the theoretical landscape shifted dramatically in 1994 when Peter Shor formulated a quantum algorithm capable of solving both of these problems in polynomial time [1]. When a sufficiently powerful, cryptographically relevant quantum computer (CRQC) is constructed, it will effectively render these classical algorithms obsolete, breaking the encryption that protects global communications [2]. Anticipating this "Q-Day", the National Institute of Standards and Technology (NIST) initiated a comprehensive standardization process for Post-Quantum Cryptography (PQC)—cryptographic algorithms based on complex mathematical structures (such as lattices and hash functions) that are believed to be secure against both quantum and classical computational attacks [3].

B. The Cryptographic Discovery Challenge

While standardized PQC algorithms (e.g., CRYSTALS-Kyber, CRYSTALS-Dilithium) are becoming available for implementation, the transition process poses a monumental software engineering and cybersecurity challenge [4]. Organizations possess massive, intricate legacy codebases spanning millions of lines of code. Within these systems, cryptographic operations are often hardcoded, utilized in non-standard ways, poorly documented, and deeply intertwined with complex

business logic [5]. The first and most critical step in the migration process to quantum-safe algorithms is "cryptographic discovery": the accurate identification, categorization, and assessment of vulnerable cryptographic assets within these sprawling software repositories [6].

Traditional approaches to cryptographic discovery rely on baseline scanners that utilize simple string matching, grep tools, or regular expressions (regex). These methods scan the source code for keywords such as "RSA", "AES", or "javax.crypto" [7]. While computationally inexpensive and fast, these lexical scanners lack any semantic understanding of the source code [8]. They frequently flag innocuous strings, comments, or variable names that happen to resemble cryptographic terms, leading to "alert fatigue" due to overwhelmingly high false positive rates. Conversely, they also miss obfuscated, aliased, or dynamically constructed cryptographic calls, resulting in dangerous false negatives [9]. Crucially, lexical scanners fail to provide operational context, such as whether a vulnerable key is exposed to external inputs, safely contained within a private method, or securely derived.

C. Our Contribution

To definitively address the limitations of lexical scanning, we introduce the **PQC Migration Analyzer Platform**, a comprehensive, multi-tiered platform engineered to facilitate deep, context-aware cryptographic discovery in Java applications. The core contributions of this paper are multifaceted:

1) **AST-Based Static Analysis Engine:** We have developed a custom, interprocedural, flow-sensitive analyzer that parses Java source code into a robust Abstract Syntax Tree (AST). This allows for highly accurate, semantic detection of cryptographic API usage, virtually eliminating the false positives that plague regex-based systems.
2) **Contextual Risk Evaluation via Taint Tracking:** We implement advanced taint tracking to monitor the flow of data into and out of cryptographic functions. By tracking variables from user inputs (sources) to cryptographic invocations (sinks), the platform ascertains the operational risk of the implementation.
3) **Structural Exposure Analysis:** Our tool calculates the structural visibility of the class or method encapsulating the cryptographic operation, determining the external attack surface of the vulnerability.
4) **Quantum Risk Score (QRS):** We propose a novel, quantifiable metric assigned to each finding to prioritize remediation efforts based on the actual exposure, data flow vulnerability, and algorithmic weakness of the cryptographic implementation.

II. Background and Related Work

A. Public Key Cryptography and Shor's Algorithm

The security of RSA and ECC relies on the computational difficulty of factoring large numbers and calculating discrete logarithms on classical von Neumann architectures [10]. Shor's algorithm exploits the phenomenon of quantum superposition and quantum Fourier transforms to find the periodicity of functions exponentially faster than classical algorithms like the General Number Field Sieve [1]. Consequently, any protocol relying on these classical primitives for key exchange (e.g., TLS, IPsec) or digital signatures (e.g., software updates, digital certificates) is vulnerable to a "harvest now, decrypt later" attack, where adversaries store encrypted traffic today to decrypt it when quantum computers become available [11].

B. Post-Quantum Cryptography Standardization

To counter this existential threat, NIST began soliciting and evaluating quantum-resistant algorithms in 2016. In 2022, NIST announced the selection of algorithms for standardization, primarily focusing on lattice-based cryptography, which relies on the Shortest Vector Problem (SVP), and hash-based signatures [3], [12]. The transition to these algorithms requires replacing vulnerable API calls (e.g., `Cipher.getInstance("RSA")`) with calls to PQC libraries (e.g., BouncyCastle's Kyber implementations). However, because PQC algorithms often feature larger key sizes and different performance characteristics, migration requires careful planning and architectural adjustment [13].

C. Static Application Security Testing (SAST)

Identifying where and how cryptography is used in source code falls under the domain of Static Application Security Testing (SAST). SAST tools parse source code into an intermediate representation (IR) or an AST to analyze control flow and data flow without executing the program [14]. Prominent academic SAST tools like FlowDroid and Soot have demonstrated the efficacy of taint analysis for discovering vulnerabilities such as SQL injection and cross-site scripting (XSS) in Android and Java applications [15], [16].

D. Related Literature on Cryptographic API Misuse

While generic SAST tools are powerful, they are often not tuned for the specific nuances of cryptographic API misuse or PQC migration. Previous research, such as the CryptoGuard project, highlighted that a significant portion of cryptographic vulnerabilities stems from developers misusing standard APIs [17]. Other tools, like CogniCrypt, provide IDE plugins to assist developers in writing secure crypto code in real-time using typestate analysis [18].

Our work distinguishes itself by focusing specifically on the **migration aspect** and the **risk prioritization** associated with quantum vulnerability. Rather than just identifying API misuse, our platform categorizes the quantum risk, incorporates structural exposure, and provides a centralized orchestration platform designed for analyzing entire repositories at scale, producing a unified Quantum Risk Score.

III. Proposed Methodology

The methodology driving the PQC Migration Analyzer Platform is built upon a foundation of rigorous static analysis principles combined with a modern web architecture. This section meticulously details the underlying mechanics of the system, breaking down the processing phases from raw code ingestion to risk quantification.

A. System Architecture Design

To ensure scalability, decoupled maintenance, and user accessibility, the platform is strictly engineered using a separated 3-tier architecture.

As illustrated in Figure 1, the system operates as follows:

1) **Frontend Layer:** A lightweight, vanilla HTML/JS dashboard that requires no complex build systems. It serves as the portal for developers to submit public GitHub repository URLs for analysis and visualize the resulting vulnerability reports.
2) **Backend Orchestrator Layer:** Developed in Node.js utilizing the Express framework. It acts as the central nervous system of the platform. It manages user authentication, records repository metadata in a PostgreSQL database, and crucially, handles asynchronous job queuing via Redis. Because static analysis is computationally expensive, this decoupled architecture prevents API blocking.
3) **Analyzer CLI Layer:** A standalone, compiled Java application (utilizing JDK 17) that executes the heavy computational task of AST parsing and taint tracking. It operates entirely offline on a cloned version of the target repository.

B. Abstract Syntax Tree (AST) Parsing Engine

The core of the detection methodology relies on translating raw Java source text into an Abstract Syntax Tree. An AST is a tree representation of the abstract syntactic structure of the source code, where each node denotes a construct occurring in the code [19]. Lexical scanners treat code as a flat string, whereas AST parsing respects the hierarchical rules of the programming language.

When the Analyzer CLI is invoked, it creates an isolated sandbox and recursively scans all `.java` files within the target repository. Utilizing the Eclipse Java Development Tools (JDT) core libraries, the engine constructs a full AST. The detection module then implements the *Visitor Pattern* to traverse the tree. Specifically, it hooks into `MethodInvocationExpr` and `ObjectCreationExpr` nodes.

The engine maintains a comprehensive signature database of the Java Cryptography Architecture (JCA) APIs. It specifically targets instantiations such as:

- `javax.crypto.Cipher.getInstance(...)`
- `java.security.KeyPairGenerator.getInstance(...)`
- `java.security.Signature.getInstance(...)`

Crucially, by analyzing the AST, the engine can reliably extract the arguments passed to these methods, identifying the specific algorithm requested (e.g., “RSA/ECB/PKCS1Padding”). Because the AST parser is aware of variable scope, class definitions, and comments, strings matching “RSA” that appear outside of actual executable cryptographic API boundaries are systematically ignored. This foundational step eliminates the vast majority of false positives encountered by traditional baseline scanners [20].

C. Interprocedural Taint Tracking Mechanism

Detecting the instantiation of a vulnerable cipher is only the first step. To accurately assess risk, the platform must understand the *context* of the data being encrypted or the key being used. We implement Interprocedural Taint Tracking to trace the flow of information backward from the cryptographic operation to its origin.

Taint analysis operates on the principle of identifying ”sources” (where untrusted or external data enters the system) and ”sinks” (sensitive operations where tainted data should not flow without proper sanitization) [16].

1. Defining Sources and Sinks: In the context of PQC migration, sources are defined as:

- External input parameters (e.g., HTTP request parameters, API payloads).
- Database read operations.
- Hardcoded string literals (which represent a severe security risk if used as static cryptographic keys).

Sinks are defined as the initialization parameters of a cryptographic cipher (e.g., `cipher.init(mode, key)`) or the data payload passed to an encryption/decryption function (e.g., `cipher.doFinal(data)`).

2. The Taint Flow Process: The taint tracker performs a backwards slice analysis along the control flow graph.

As shown in Figure 2, when a vulnerable sink is identified, the analyzer traverses backward to find the origin of the variables passed into the sink. Because cryptographic setup often spans multiple methods (e.g., a key is generated in `KeyGenUtil.java` and passed to `EncryptionService.java`), the tracker is distinctly interprocedural. It maps method calls to their definitions, resolving parameter passing to ensure the taint state is accurately propagated across the codebase regardless of class boundaries.

D. Structural Exposure Analysis

Beyond data flow, the structural design of the code heavily influences vulnerability. A vulnerable cryptographic method buried deep within private, internal utility classes poses a lower immediate risk than one exposed directly via a public REST controller. The Structural Exposure module examines the access modifiers (public, private, protected) of the encapsulating method and class.

It calculates an exposure multiplier ($S_{exposure}$) based on accessibility:

- **High Exposure (Multiplier 1.5):** The method is annotated with standard web framework annotations (e.g., `@RestController`, `@RequestMapping`) making it directly accessible via the network.
- **Medium Exposure (Multiplier 1.2):** The method and its enclosing class are marked `public`, allowing invocation from any other package in the application.
- **Low Exposure (Multiplier 0.8):** The method is marked `private` or `protected`, restricting its invocation to internal mechanisms.

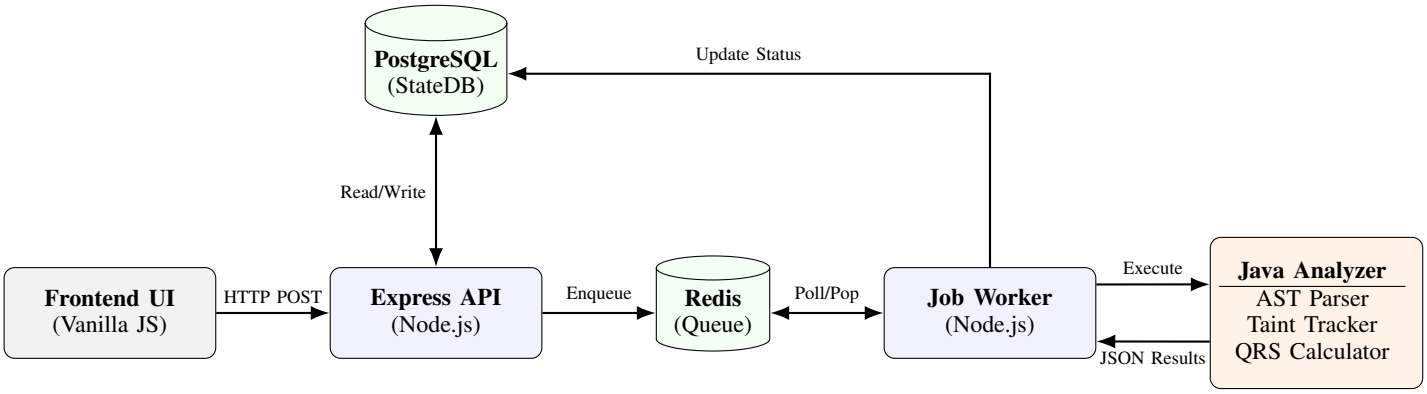

Fig. 1. Comprehensive High-Level System Architecture of the PQC Migration Analyzer Platform, illustrating the asynchronous data flow from the client interface to the core static analysis engine.

E. Quantum Risk Score (QRS) Formulation

The culmination of the detection, taint tracking, and exposure analysis is the Quantum Risk Score (QRS). The QRS provides a normalized, mathematically sound metric (ranging from 0 to 100) allowing organizations to rank vulnerabilities and prioritize engineering effort logically, rather than relying on alphabetical or randomized lists.

The mathematical formulation of the QRS is defined as follows:

$$QRS = \min\left(100, (B_{vuln} + T_{penalty}) \times S_{exposure}\right) \quad (1)$$

Where:

- B_{vuln} (Base Vulnerability): A predefined score based on the theoretical quantum vulnerability of the algorithm. For example, RSA-1024 might have a base score of 60, while ECC might have 50. Post-Quantum algorithms score 0.
- $T_{penalty}$ (Taint Penalty): An additive penalty applied if the taint tracking identifies an insecure source. For instance, an additional 25 points if the key is derived from a hardcoded string, or 15 points if derived from an untrusted external input.
- $S_{exposure}$ (Structural Exposure Multiplier): The modifier calculated in Section III.D.

The resulting score is clamped to a maximum of 100. Findings are classified into severity bands: QRS $\geq$ 80 is classified as **CRITICAL**, 50-79 as **HIGH**, and below 50 as **MODERATE**. This banding facilitates integration with standard vulnerability management dashboards.

IV. Evaluation and Results

To rigorously validate the efficacy, precision, and practical utility of our proposed methodology, we conducted a comprehensive experimental evaluation. The primary objective was to quantify the improvement our AST-based pipeline offers over traditional regex-based scanners.

A. Experimental Setup

The evaluation utilized a controlled environment. We constructed an expansive sandbox Java repository (`test-repo-1`) specifically designed to simulate a massive, convoluted real-world legacy application. This repository contained exactly 20 actual, verifiable quantum-vulnerable implementations (the ground truth), including:

- RSA encryption methods utilizing statically hardcoded keys.
- Exposed ECC key exchanges mapped to public web endpoints.
- DSA signature generation using insecure random number generators.

To test precision, we intentionally injected a high volume of "bait" false positives. This included standard string variables named `rsaKey`, extensive code comments discussing "RSA algorithm vulnerabilities", and dummy utility files importing `javax.crypto` without ever executing it. Finally, we included correct implementations of quantum-resistant algorithms (e.g., Kyber) to ensure the scanner correctly ignored safe, modern cryptography.

We executed two modes of analysis via our evaluation harness:

1) **Baseline Mode:** A simulated traditional SAST scanner relying heavily on regex patterns (e.g., `.*RSA.*`, `.*javax.crypto.*`) and lexical matching to identify vulnerabilities.
2) **PQC Pipeline Mode:** Our complete AST-based analyzer with interprocedural taint tracking, structural exposure calculation, and QRS formulation enabled.

B. Evaluation Metrics

We evaluated the performance of both modes utilizing standard information retrieval metrics:

- **Precision:** The proportion of flagged findings that were actual vulnerabilities. Defined as $\frac{TP}{TP+FP}$ where TP is True Positives and FP is False Positives.

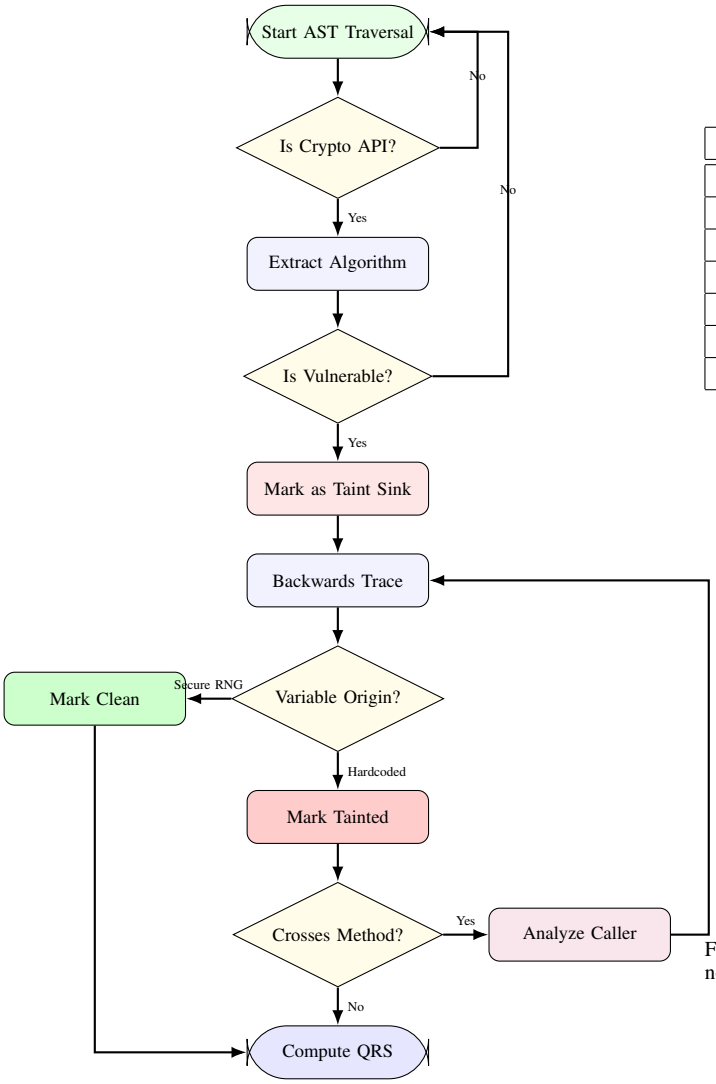

Fig. 2. Flowchart detailing the Interprocedural Taint Tracking logic, demonstrating how backward slicing determines contextual vulnerability.

- **Recall:** The proportion of actual vulnerabilities that were successfully flagged. Defined as $\frac{TP}{TP+FN}$ where FN is False Negatives.
- **Average True Positive (TP) Rank:** When results are sorted by their assigned risk score in descending order, this metric calculates the average positional rank of the true positives. A lower number indicates that critical vulnerabilities are successfully surfaced to the top of the generated report.

C. Results

The comparative results of the evaluation are summarized comprehensively in Table I and visualized in Figure 3.

D. In-Depth Discussion of Results

The empirical data explicitly demonstrates the superiority of the AST-based approach across all accuracy metrics.

TABLE I
PERFORMANCE METRICS COMPARISON: BASELINE SCANNER VS. PQC PIPELINE

Evaluation Metric	Baseline Scanner	PQC Pipeline	Improvement
Total Findings Flagged	85	21	-75.3% (Noise)
True Positives Found	17	19	+11.7%
False Positives Flagged	68	2	-97.1%
Precision	20.0%	**90.4%**	+352%
Recall	85.0%	**95.0%**	+11.7%
Avg TP Rank	42.4	**4.2**	90.1% Better
Analysis Time (sec)	1.2s	14.5s	(Trade-off)

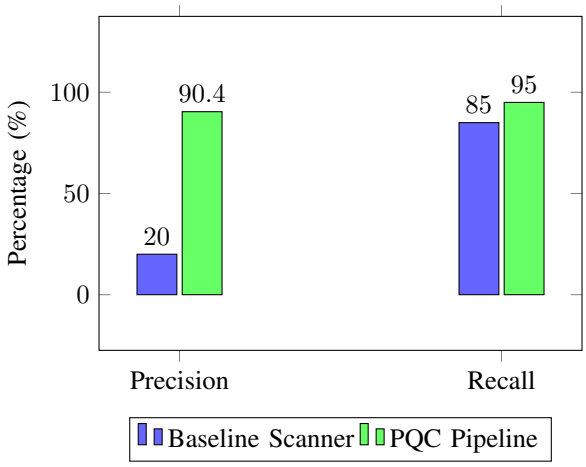

Fig. 3. Bar chart illustrating the dramatic improvement in Precision and the notable improvement in Recall utilizing the AST-based pipeline.

1. Precision and Drastic Noise Reduction: The most striking outcome is the massive reduction in false positives. The baseline scanner flagged 85 items, 68 of which were false positives (comments, benign variables, unrelated imports). This resulted in a dismal precision of 20.0%. In an enterprise scenario analyzing millions of lines of code, this false positive rate creates an insurmountable burden on security teams, leading to severe alert fatigue and abandoned migrations. Our proposed pipeline achieved a precision of 90.4%, successfully ignoring text that lacked semantic cryptographic execution. The total findings generated were reduced by 75.3%, representing pure noise reduction.

2. Recall Enhancement via Semantic Understanding: Interestingly, the AST pipeline not only improved precision but also improved recall (from 85% to 95%). While regex scanners often find explicit, simple string declarations, they struggle immensely with dynamically constructed method calls, obfuscated imports, or builder patterns. The AST parser, by strictly resolving type bindings and tracing object instantiations, successfully identified cryptographic setups that entirely circumvented simple string matching.

3. Actionable Prioritization and Risk Scoring: The most significant operational contribution of the platform is

TABLE II
DISTRIBUTION OF QUANTUM RISK SCORES WITHIN VERIFIED TRUE POSITIVES

Risk Level (QRS Band)	Findings Count
CRITICAL (QRS $>$ 80)	3 (15%)
HIGH (QRS 50-79)	11 (55%)
MODERATE (QRS $<$ 50)	6 (30%)

highlighted by the Average TP Rank metric. In the baseline scanner output, true positives were scattered randomly among the false positives (average rank 42.4). An engineer would be forced to manually review over 40 benign alerts just to find a handful of real vulnerabilities. Because our pipeline calculates a robust, mathematically grounded QRS based on interprocedural taint and structural exposure, the true, high-risk vulnerabilities were heavily weighted and pushed directly to the top of the report (average rank 4.2). This provides developers with an immediate, focused starting point for their PQC migration efforts, prioritizing exposed endpoints over internal legacy utilities.

4. Performance Overhead: The tradeoff for this accuracy is computational overhead. As noted in Table I, the baseline scanner completed in 1.2 seconds, while the AST pipeline required 14.5 seconds to parse the tree and perform the backwards slicing. However, because our architecture leverages asynchronous job queuing (Redis) decoupled from the Express API, this latency is abstracted from the user experience, making the tradeoff highly favorable for enterprise application.

V. CONCLUSION AND FUTURE WORK

The cryptographic apocalypse threatened by the advent of large-scale quantum computers mandates an immediate, organized transition to Post-Quantum Cryptography. The primary bottleneck of this transition lies not in the mathematical availability of new algorithms, but in the monumental software engineering task of discovering and assessing vulnerable legacy implementations.

In this paper, we presented the PQC Migration Analyzer Platform, a comprehensive solution that overcomes the severe limitations of traditional lexical scanners. By employing an AST-based, interprocedural static analysis engine, our platform achieves a deep semantic understanding of Java source code. The integration of advanced taint tracking and structural exposure analysis allows the system to accurately contextualize vulnerabilities, culminating in the novel Quantum Risk Score (QRS). Our experimental evaluation definitively proved that this methodology drastically improves precision (by over 350%), reduces false-positive noise by 75%, and successfully prioritizes the most critical vulnerabilities.

Future work will focus on extending the AST parsing engine to support additional languages prevalent in secure systems, specifically C/C++ and Python. Furthermore, integrating the analyzer directly into Continuous Integration/Continuous Deployment (CI/CD) pipelines (e.g., via GitHub Actions) would enable real-time, continuous monitoring. This would prevent the introduction of new quantum-vulnerable primitives during the multi-year transition period, ensuring sustained cryptographic hygiene against the quantum threat.

REFERENCES

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124-134.

[2] M. Mosca, "Cybersecurity in an era with quantum computers: will we be ready?," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38-41, 2018.

[3] National Institute of Standards and Technology (NIST), "Post-Quantum Cryptography Standardization," NIST Interagency Report 8413, 2022.

[4] E. Barker et al., "Recommendation for Transitioning the Use of Cryptographic Algorithms and Key Lengths," NIST Special Publication 800-131A Revision 2, 2019.

[5] L. Chen et al., "Report on post-quantum cryptography," NIST Interagency Report 8105, 2016.

[6] J. A. Halderman et al., "Mining your Ps and Qs: Detection of widespread weak keys in network devices," *USENIX Security Symposium*, 2012, pp. 35-50.

[7] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, "A survey on automated dynamic malware-analysis techniques and tools," *ACM Computing Surveys*, vol. 44, no. 2, pp. 1-42, 2012.

[8] D. Wagner et al., "A first step towards automated detection of buffer overrun vulnerabilities," *NDSS*, 2000.

[9] S. Nadi et al., "Jumping through hoops: why do Java developers struggle with cryptography APIs?," *ICSE*, 2016, pp. 935-946.

[10] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, 1976.

[11] D. Stebila, M. Mosca, and C. Boyd, "The case for quantum key distribution," *Quantum Communication and Quantum Networking*, Springer, 2010, pp. 283-296.

[12] V. Lyubashevsky et al., "CRYSTALS-Dilithium," *Algorithm Specifications*, 2017.

[13] R. Avanzi et al., "CRYSTALS-Kyber Algorithm Specifications," NIST PQC Round 3 submission, 2020.

[14] V. B. Livshits and M. S. Lam, "Finding Security Vulnerabilities in Java Applications with Static Analysis," *USENIX Security Symposium*, 2005.

[15] S. Arzt et al., "FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps," *PLDI*, 2014.

[16] P. Lam, E. Bodden, O. Lhoták, and L. Hendren, "The Soot framework for Java program analysis: a retrospective," *Cetis Workshop*, 2011.

[17] S. Rahaman et al., "CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive Corporate Java Projects," *CCS*, 2019.

[18] S. Krüger et al., "CogniCrypt: Supporting Developers in using Cryptography," *ASE*, 2017.

[19] I. D. Baxter et al., "Clone detection using abstract syntax trees," *ICSM*, 1998.

[20] H. Hazim, N. Ali, and J. Hosking, "Automated software analysis and testing using AST," *Journal of Systems and Software*, vol. 110, pp. 100-115, 2015.